Sprawozdanie z projektu zaliczeniowego pt. „Scena z Minecrafta” z przedmiotu Grafika komputerowa

Wykonali: Drożyński Piotr, Bednarczyk Jakub, Bednarczyk Mikołaj

1. Wstęp

Jako temat naszego projektu wybraliśmy **Scenkę z popularnej gry Minecraft.** Stworzony przez nas model przedstawia scenę ze wspomnianej gry na której widzimy budynek/domek, drzewo, Creepera którym można się poruszać, głowę świni którą można skalować (powiększać i pomniejszać) oraz słońce animowane, które przemieszcza się po niebie oświetlając inne obiekty. Można zmieniać szybkość przemieszczania się słońca Tło i niebo zostały wykonane przez teksturowanie sześcianu grafiką skybox. Nasz projekt posiada również Możliwość poruszania się kamerą, model oświetlenie Phonga i możliwość włączenia efektu rozmycia Gaussa.

2. Realizacja

I. Inicjalizacja i konfiguracja OpenGL, tworzenie okna

Scena 3D została zainicjalizowana za pomocą GLFW (tworzenie okna, obsługa zdarzeń) oraz GLAD (funkcje OpenGL). Ustawiono profil OpenGL 3.3 Core. Zainicjalizowano test gębokości (glEnable(GL_DEPTH_TEST)) oraz obsługę myszy i klawiatury.

```
// Initialize GLFW
glfwInit();
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);

// Create window
GLFWwindow* window = glfwCreateWindow(width, height, "Maincamp - Projekt", NULL, NULL);
if (window == NULL)
{
    std::cout << "Failed to create GLFW window" << std::endl;
    glfwTerminate();
    return -1;
}
glfwMakeContextCurrent(window);
glfwSetFramebufferSizeCallback(window, framebuffer_size_callback);
glfwSetCursorPosCallback(window, mouse_callback);
glfwSetScrollCallback(window, scroll_callback);
glfwSetInputMode(window, GLFW_CURSOR, GLFW_CURSOR_DISABLED);

// Load GLAD
if (!gladLoadGLLoader((GLADloadproc)glfwGetProcAddress))
{
    std::cout << "Failed to initialize GLAD" << std::endl;
    return -1;
}

// Configure global OpenGL state
glEnable(GL_DEPTH_TEST);
```

II. Kamera i sterowanie

Zaimplementowano kamerę typu FPS z rzutowaniem perspektywicznym. Kamera obsługuje: Poruszanie się: klawisze W, A, S, D, spacja (góra), shift (dół), Obrót kamerą: mysz, Zoom: scroll myszy

Macierze rzutowania i widoku tworzone są z wykorzystaniem biblioteki GLM:

glm::mat4 projection = glm::perspective(glm::radians(fov), (float)width/height, 0.1f, 200.0f);

glm::mat4 view = glm::lookAt(cameraPos, cameraPos + cameraFront, cameraUp);

plik Camer.cpp:

```
#include "Camera.h"

Camera::Camera(int width, int height, glm::vec3 position)
{
    Camera::width = width;
    Camera::height = height;
    Position = position;
}

void Camera::Matrix(Shader& shader, const char* uniform)
{
    // Eksportuje macierz kamery
    glUniformMatrix4fv(glGetUniformLocation(shader.ID, uniform), 1, GL_FALSE,
        glm::value_ptr(cameraMatrix));

}
void Camera::Inputs(GLFWwindow* window) { ... }

void Camera::updateMatrix(float FOVdeg, float nearPlane, float farPlane)
{
    glm::mat4 view = glm::mat4(1.0f);
    glm::mat4 projection = glm::mat4(1.0f);
    view = glm::lookAt(Position, Position + Orientation, Up);
    projection = glm::perspective(glm::radians(FOVdeg), (float)width / height,
        nearPlane, farPlane);
    cameraMatrix = projection * view;
}
```

III. Modelowanie sceny 3D i renderowanie obiektów 3D

Cała scena zbudowana jest na bazie sześcianów, które współdzielą dane wierzchołków. Wykorzystano instancjonowanie oraz transformacje macierzy model do rozmieszczania obiektów.

Przykładowy kod, tutaj akurat dla obiektów drzewo i Creeper:

```
void renderTree(Shader& shader, Texture& woodTex, Texture& leavesTex)
{
    // Trunk (4 blocks tall)
    for (int i = 0; i < 4; i++) {
        renderCube(shader, woodTex, glm::vec3(2.0f, 0.5f + i, 0.0f));
    }

    // Leaves on top
    renderCube(shader, leavesTex, glm::vec3(2.0f, 3.5f, 0.0f));
    renderCube(shader, leavesTex, glm::vec3(2.0f, 4.5f, 0.0f));
    renderCube(shader, leavesTex, glm::vec3(1.0f, 3.5f, 0.0f));
    renderCube(shader, leavesTex, glm::vec3(3.0f, 3.5f, 0.0f));
    renderCube(shader, leavesTex, glm::vec3(2.0f, 3.5f, 1.0f));
    renderCube(shader, leavesTex, glm::vec3(2.0f, 3.5f, -1.0f));
}

void renderHouse(Shader& shader, Texture& houseTex) { ... }

void renderCreeper(Shader& shader, Texture& creeperTex)
{
    glm::mat4 model = glm::mat4(1.0f);
    model = glm::translate(model, creeperPos);
    model = glm::rotate(model, atan2(creeperFront.x, creeperFront.z), glm::vec3(0.0f, 1.0f, 0.0f));

    // Bottom block
    glm::mat4 bottomModel = model;
    bottomModel = glm::translate(bottomModel, glm::vec3(0.0f, 0.5f, 0.0f));
    glUniformMatrix4fv(glGetUniformLocation(shader.ID, "model"), 1, GL_FALSE, &bottomModel[0][0]);
    creeperTex.Bind();
    glDrawArrays(GL_TRIANGLES, 0, 36);

    // Top block
    glm::mat4 topModel = model;
    topModel = glm::translate(topModel, glm::vec3(0.0f, 1.5f, 0.0f));
    glUniformMatrix4fv(glGetUniformLocation(shader.ID, "model"), 1, GL_FALSE, &topModel[0][0]);
    creeperTex.Bind();
    glDrawArrays(GL_TRIANGLES, 0, 36);
}
```

IV. Rzutowanie perspektywiczne

Zastosowano kamerę 3D z rzutowaniem perspektywicznym:

Macierz projection generowana za pomocą glm::perspective

Macierz view generowana przez glm::lokat

Możliwość zmiany pola widzenia (FOV) za pomocą scrolla myszy

```
glm::mat4 projection = glm::perspective(glm::radians(100.0f), (float)width / (float)height, 0.1f, 200.0f);
glm::mat4 view = glm::lookAt(cameraPos, cameraPos + cameraFront, cameraUp);
```

V. Interakcja z użytkownikiem

Zaimplementowano rozbudowany system sterowania:

- **Klawiatura:**
  - **Poruszanie kamerą (WASD, spacja, shift)**
  - **Sterowanie pozycją Creepera (strzałki)**
  - **Zmiana prędkości światła (+/-)**
  - **Skalowanie głowy świni (klawisze numeryczne +/-)**
  - **Włączanie/wyłączanie efektu blur (B)**
- **Mysz:**
  - **Sterowanie kierunkiem kamery**
  - **Zmiana pola widzenia (scroll)**

VI. Oświetlenie

Zastosowano model oświetlenia Phonga. Światło porusza się po okręgu (imitacja słońca). Wykorzystano składniki ambient, diffuse i specular:

```
struct Material {
    float shininess;
};

uniform Material material;

void main()
{
    // Ambient
    float ambientStrength = 0.1;
    vec3 ambient = ambientStrength * lightColor;

    // Diffuse
    vec3 norm = normalize(Normal);
    vec3 lightDir = normalize(lightPos - FragPos);
    float diff = max(dot(norm, lightDir), 0.0);
    vec3 diffuse = diff * lightColor;

    // Specular
    float specularStrength = 0.5;
    vec3 viewDir = normalize(viewPos - FragPos);
    vec3 reflectDir = reflect(-lightDir, norm);
    float spec = pow(max(dot(viewDir, reflectDir), 0.0), material.shininess);
    vec3 specular = specularStrength * spec * lightColor;

    vec3 result = (ambient + diffuse + specular) * texture(texture_diffuse1, TexCoords).rgb;
    FragColor = vec4(result, 1.0);
}
```

VII. Shaderowanie / Cieniowanie (GLSL)

Vertex Shader (default.vert):

- Przekształca pozycje wierzchołków do przestrzeni ekranowej
- Oblicza normalne po transformacjach

```
void main()
{
    FragPos = vec3(model * vec4(aPos, 1.0));
    Normal = mat3(transpose(inverse(model))) * aNormal;
    TexCoords = aTexCoords;

    gl_Position = projection * view * vec4(FragPos, 1.0);
}
```

Fragment Shader (default.frag):

- Implementuje model Phonga
- Pobiera kolor z tekstury

VIII. Teksturowanie

Wszystkie obiekty korzystają z dedykowanych tekstur w formacie PNG.

Tekstury ładowane i aktywowane za pomocą klasy Texture:

```
Texture::Texture(const char* image, GLenum texType, GLenum slot, GLenum format, GLenum pixelType)
{
    type = texType;

    int widthImg, heightImg, numColCh;
    stbi_set_flip_vertically_on_load(true);

    // Wymuszenie 4 kanałów RGBA
    unsigned char* bytes = stbi_load(image, &widthImg, &heightImg, &numColCh, 4);

    if (!bytes)
    {
        std::cerr << "Failed to load texture: " << image << std::endl;
        ID = 0;
        return;
    }

    glGenTextures(1, &ID);
    glActiveTexture(slot);
    glBindTexture(texType, ID);

    glTexParameteri(texType, GL_TEXTURE_MIN_FILTER, GL_NEAREST_MIPMAP_LINEAR);
    glTexParameteri(texType, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
    glTexParameteri(texType, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(texType, GL_TEXTURE_WRAP_T, GL_REPEAT);

    glTexImage2D(texType, 0, GL_RGBA, widthImg, heightImg, 0, GL_RGBA, pixelType, bytes);
    glGenerateMipmap(texType);

    stbi_image_free(bytes);
    glBindTexture(texType, 0);
}
```

IX. Animacje i ruch obiektów

Zaimplementowano kilka rodzajów animacji:

- **Ruch słońca** - orbitalny ruch źródła światła
- **Sterowanie Creeperem** - poruszanie się postaci klawiszami strzałek
- **Skalowanie głowy świni** - dynamiczna zmiana rozmiaru obiektu
- **Zmiana koloru nieba** - w zależności od pozycji słońca

Poruszanie się Creepera:

```
void updateCreeper(float deltaTime, GLFWwindow* window) {
    const float moveSpeed = 3.0f * deltaTime; // Base movement speed
    bool moved = false;

    if (glfwGetKey(window, GLFW_KEY_UP) == GLFW_PRESS) {
        creeperPos.z -= moveSpeed;
        creeperFront = glm::vec3(0.0f, 0.0f, -1.0f);
        moved = true;
    }
    if (glfwGetKey(window, GLFW_KEY_DOWN) == GLFW_PRESS) {
        creeperPos.z += moveSpeed;
        creeperFront = glm::vec3(0.0f, 0.0f, 1.0f);
        moved = true;
    }
    if (glfwGetKey(window, GLFW_KEY_LEFT) == GLFW_PRESS) {
        creeperPos.x -= moveSpeed;
        creeperFront = glm::vec3(-1.0f, 0.0f, 0.0f);
        moved = true;
    }
    if (glfwGetKey(window, GLFW_KEY_RIGHT) == GLFW_PRESS) {
        creeperPos.x += moveSpeed;
        creeperFront = glm::vec3(1.0f, 0.0f, 0.0f);
        moved = true;
    }

    // Keep creeper within bounds
    creeperPos.x = glm::clamp(creeperPos.x, -5.0f, 5.0f);
    creeperPos.z = glm::clamp(creeperPos.z, -5.0f, 5.0f);

    // Only rotate if we actually moved
    if (!moved) {
        // Keep the current facing direction
    }
```

X. Filtrowanie obrazu (Post-processing i efekty specjalne)

Zaimplementowano zaawansowany system post-processingu:

- **Framebuffer Object** (FBO) do renderowania sceny do tekstury
- **Efekt blur** z użyciem kernelu splotowego
- Możliwość dynamicznego włączania/wyłączania efektu (klawisz B)
- Shader post-processingu z jądrem Gaussa

Scena renderowana jest najpierw do Framebuffer Object (FBO), a następnie nakładany jest efekt rozmycia.

Shader Framebuffer.frag:

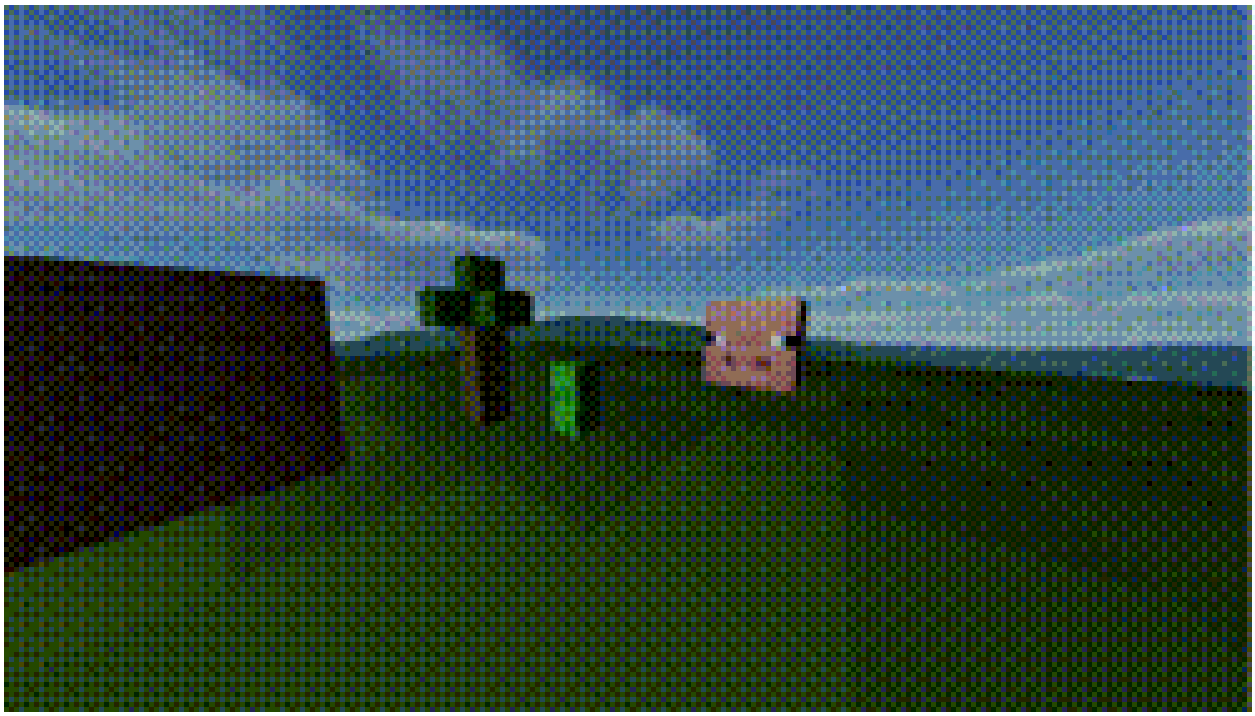
```
#version 330 core
out vec4 FragColor;
in vec2 texCoords;
uniform sampler2D screenTexture;

const float offset = 1.0 / 800.0;

vec2 offsets[9] = vec2[](
    vec2(-offset,  offset), vec2( 0.0f,  offset), vec2( offset,  offset),
    vec2(-offset,  0.0f),   vec2( 0.0f,  0.0f),   vec2( offset,  0.0f),
    vec2(-offset, -offset), vec2( 0.0f, -offset), vec2( offset, -offset)
);

float kernel[9] = float[](
    1.0 / 16, 2.0 / 16, 1.0 / 16,
    2.0 / 16, 4.0 / 16, 2.0 / 16,
    1.0 / 16, 2.0 / 16, 1.0 / 16
);

void main()
{
    vec3 color = vec3(0.0);
    for(int i = 0; i < 9; ++i)
        color += texture(screenTexture, texCoords + offsets[i]).rgb * kernel[i];
    FragColor = vec4(color, 1.0);
}
```

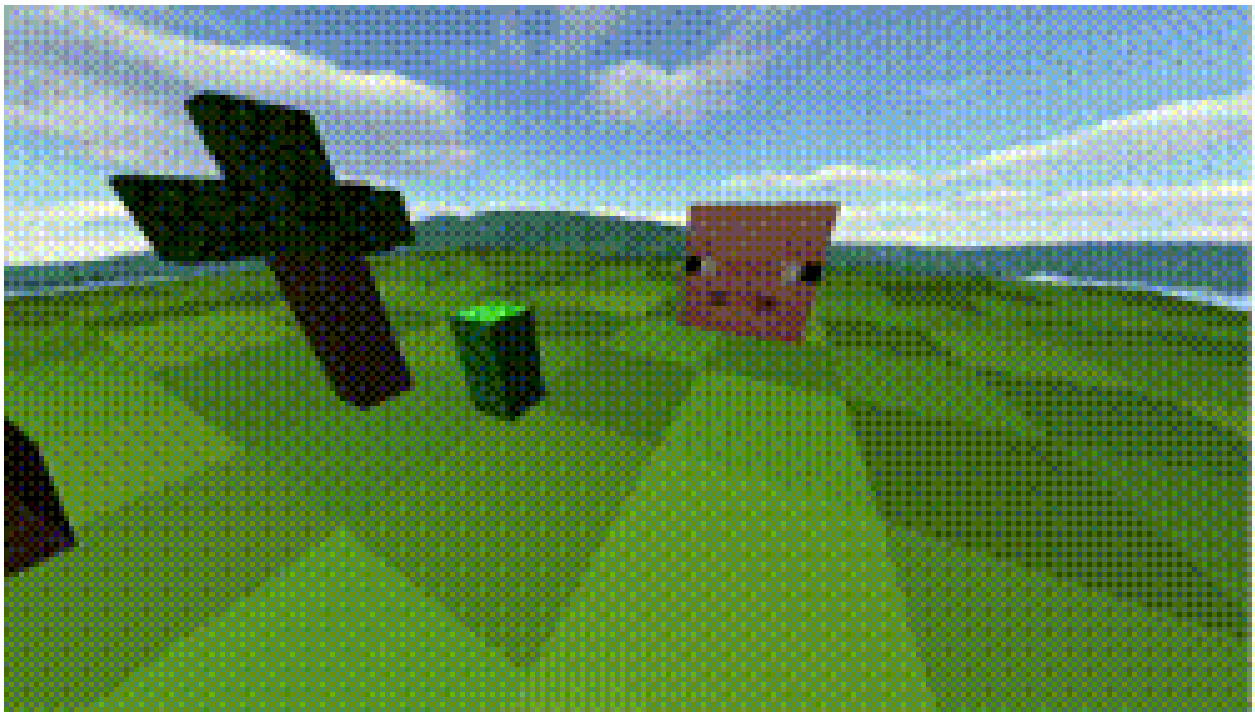
```
// --- 1. RENDERUJ SCENĘ DO FRAMEBUFFERA ---
glBindFramebuffer(GL_FRAMEBUFFER, framebuffer);
glEnable(GL_DEPTH_TEST);

glm::vec4 skyColor = glm::mix(nightColor, defaultSkyColor, sin(lightAngle));
glClearColor(skyColor.x, skyColor.y, skyColor.z, skyColor.w);
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
```

```
// 2. Render fullscreen quad with blur if enabled, else just render normally
glBindFramebuffer(GL_FRAMEBUFFER, 0); // back to default framebuffer
glDisable(GL_DEPTH_TEST);
glClear(GL_COLOR_BUFFER_BIT);

if (blurEnabled) {
    blurShader.Activate();
}
else {

    defaultScreenShader.Activate(); // shader, który renderuje teksturę bez zmian
}

glBindVertexArray(quadVAO);
glBindTexture(GL_TEXTURE_2D, textureColorbuffer);
glDrawArrays(GL_TRIANGLES, 0, 6);
```

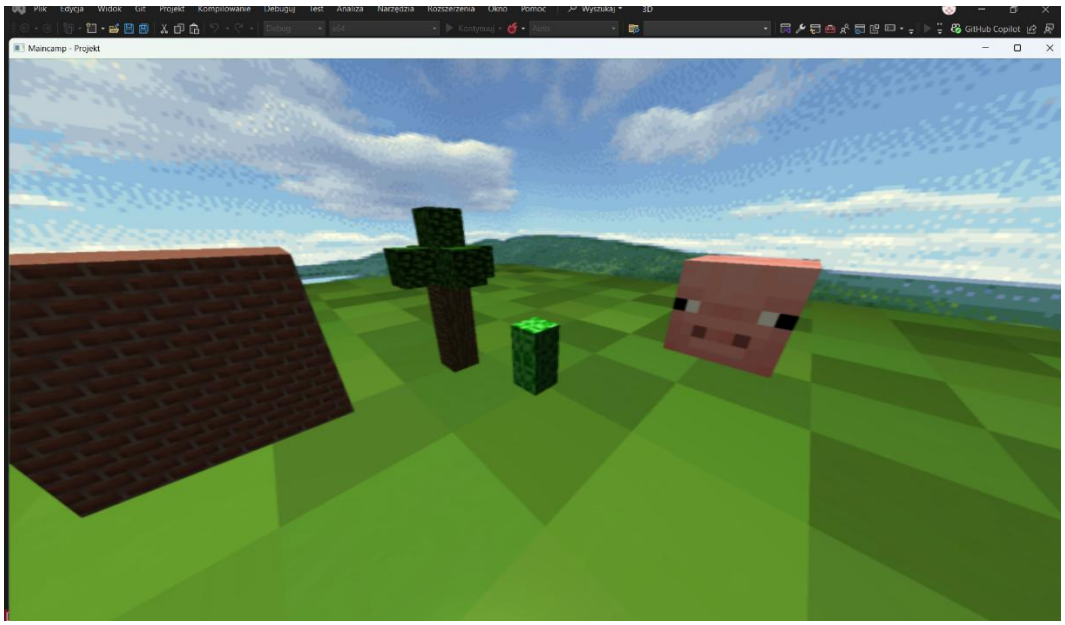
3. **Efekt końcowy**

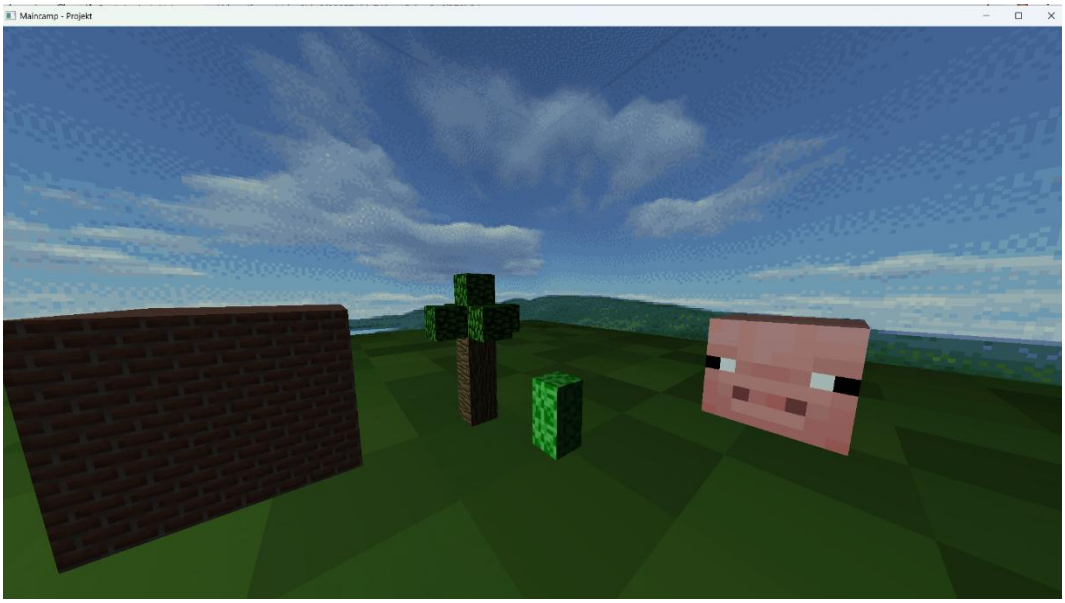
   Gify z działania naszego projektu.





Screeny z naszego projektu.





4. Wnioski

Projekt demonstruje kompleksowe wykorzystanie nowoczesnego OpenGL do tworzenia interaktywnych scen 3D. Kluczowe osiągnięcia:

1. **Modularna architektura** - System klas (VAO, VBO, EBO, Shader, Texture) zapewnia czysty i łatwy w utrzymaniu kod
2. **Efektywne wykorzystanie shaderów** - Zaawansowane efekty wizualne przy zachowaniu wydajności
3. **Interaktywność** - Bogate możliwości sterowania sceną i obiektami
4. **Realistyczne oświetlenie** - Dynamiczny system światła symulujący naturalne warunki
5. **Post-processing** - Profesjonalne efekty graficzne stosowane po renderowaniu sceny

Główne wyzwania napotkane podczas implementacji:

- Poprawne zarządzanie pamięcią buforów OpenGL
- Transformacje hierarchii obiektów
- Optymalizacja shaderów pod kątem wydajności
- Balans między realizmem a wydajnością renderowania

Projekt stanowi solidną podstawę do dalszego rozwijania zaawansowanych technik grafiki komputerowej, takich jak:

- Zaawansowane cieniowanie (PBR)
- Efekty atmosferyczne (mgła, chmury)
- Animacje szkieletowe
- Zaawansowane techniki post-processingu (SSAO, bloom)

Wszystkie pliki naszego projektu są dostępne w naszym repozytorium na GitHub pod linkiem:

https://github.com/BediGT/3D_OpenGL